Аккредитованное образовательное частное учреждение
высшего образования
«Московский финансово-юридический университет МФЮА»
(МФЮА)

КАФЕДРА «ИНФОРМАЦИОННЫЕ СИСТЕМЫ И ТЕХНОЛОГИИ»

КУРСОВАЯ РАБОТА

по дисциплине: «Разработка программных модулей»

на тему: «Разработка модуля управления каталогом товаров для интернет-магазина»

Специальность: 09.02.07
Информационные системы и программирование
Профиль: Программист

Автор работы: Чемоданов Никита Сергеевич
Курс: 4 Группа: 23ИПо8381 (ИПо-41)
Руководитель работы: Колпащиков Кирилл Александрович
Подписи и даты: _____

Киров 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ....3

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И АНАЛИЗ УПРАВЛЕНИЯ ТОВАРНЫМ КАТАЛОГОМ....5

1.1 ОСОБЕННОСТИ ВЕДЕНИЯ КАТАЛОГА ТОВАРОВ ИНТЕРНЕТ-МАГАЗИНА....5
1.2 ПОДХОДЫ К АВТОМАТИЗАЦИИ И ТРЕБОВАНИЯ К МОДУЛЮ....7
1.3 ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ....9

2 ПРОЕКТИРОВАНИЕ МОДУЛЯ УПРАВЛЕНИЯ КАТАЛОГОМ....12

2.1 ПОЛЬЗОВАТЕЛЬСКИЕ СЦЕНАРИИ И ФУНКЦИОНАЛЬНАЯ МОДЕЛЬ....12
2.2 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ И ОГРАНИЧЕНИЙ ЦЕЛОСТНОСТИ....15
2.3 АРХИТЕКТУРА ПРИЛОЖЕНИЯ И ПРОЕКТИРОВАНИЕ ИНТЕРФЕЙСА....18

3 РАЗРАБОТКА И ТЕСТИРОВАНИЕ МОДУЛЯ....22

3.1 РЕАЛИЗАЦИЯ ОПЕРАЦИЙ С ТОВАРАМИ И КАТЕГОРИЯМИ....22
3.2 РЕАЛИЗАЦИЯ ПОИСКА, ФИЛЬТРАЦИИ И ВЕБ-ИНТЕРФЕЙСА....24
3.3 ТЕСТИРОВАНИЕ, АНАЛИЗ РЕЗУЛЬТАТОВ И ОГРАНИЧЕНИЯ РЕШЕНИЯ....28

ЗАКЛЮЧЕНИЕ....33

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ....34

ПРИЛОЖЕНИЕ А. ЭКРАНЫ И РЕЗУЛЬТАТЫ ДЕМОНСТРАЦИИ....37

ВВЕДЕНИЕ

Запись о товаре приходится менять на протяжении всей работы магазина: исправлять название, назначать другую категорию, обновлять цену и доступное количество. Одного перечня позиций для этого недостаточно. При сохранении нужно отличить новый товар от уже существующего, проверить его поля и не потерять прежние сведения после ошибочного ввода. Эти операции составляют содержание разрабатываемого модуля каталога. Менеджер работает с ним в браузере; те же записи доступны через JSON API.

Основой для разработки служит выданное задание по дисциплине «Разработка программных модулей». В нём перечислены операции с товарами, поиск, фильтрация, сортировка и сохранение данных. Задание ссылается на ГОСТ 19.201–78 [1]. Для структуры записки и её оформления использованы методические указания МФЮА [2]. В границы работы входят товары и категории. Оформление покупок, расчёты и доставка относятся к другим частям интернет-магазина и здесь не реализованы.

Разбор программы опирается на несколько групп источников. Описание классов и словарей Python связано с представлением записей и значений формы [3]. Разделение обработчиков запросов и прикладных операций рассмотрено по книге Architecture Patterns with Python [5]. Материалы о функциональных и модульных тестах использованы для выбора уровней проверки [6; 8]. Эти положения далее сопоставляются с конкретными файлами, запросами и результатами работы каталога.

Цель работы — разработать и проверить модуль управления каталогом товаров интернет-магазина. Для этого необходимо определить требования к товарам и категориям, выбрать средства разработки, спроектировать базу данных и интерфейс, реализовать операции каталога. Проверка результата включает подготовку демонстрационных данных, испытание допустимых и ошибочных запросов, а также описание ограничений программы.

Объект исследования — ведение товарного каталога интернет-магазина. Предмет исследования составляют модель товаров и категорий, правила изменения записей и обработка запросов к ним. Для примеров взяты канцелярские товары: ручки, бумага, тетради и другие принадлежности. Каталог сформирован программно; он не является выгрузкой действующего предприятия. Цены, количество и описания в нём заданы для демонстрации разных состояний данных.

В работе сопоставляются способы ведения каталога, строится модель данных и проверяется реализация пользовательских действий. При испытании важен не только ответ сервера. После сохранения товар читается повторно, а после отклонённого изменения сравнивается с исходной записью. Такой порядок позволяет проверить, что сообщение интерфейса соответствует состоянию базы. Экономический эффект от внедрения и затраты времени менеджера не оценивались.

Первая глава посвящена составу каталога, требованиям и используемым средствам разработки. Во второй разобраны сценарии работы, таблицы базы и передача запроса между компонентами. В третьей рассмотрены функции приложения и результаты их проверки. Приложение содержит снимки реальных страниц. Вместе с запиской в репозитории находятся программа, загрузчик примеров, команды запуска и автоматические тесты.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И АНАЛИЗ УПРАВЛЕНИЯ ТОВАРНЫМ КАТАЛОГОМ

1.1 ОСОБЕННОСТИ ВЕДЕНИЯ КАТАЛОГА ТОВАРОВ ИНТЕРНЕТ-МАГАЗИНА

В каталоге учитывается товарная позиция с собственным артикулом. У неё есть название, категория, цена, описание и остаток. Название помогает найти запись, но не обязано отличаться от названий других товаров. Например, две ручки могут называться одинаково и при этом иметь разные цвета. Если их количество нужно учитывать отдельно, для каждого варианта создаётся своя позиция. Общие сведения о категории при этом не копируются в каждый товар.

У записи имеются два разных обозначения: id и артикул. База назначает id при создании; по нему строятся адрес карточки и связь с категорией. Артикул вводится в форме и может быть исправлен. Поэтому после замены артикула адрес товара остаётся прежним. При проверке нового значения из поиска дубликата исключается сама редактируемая запись: сохранить её прежний артикул допустимо, занять артикул другого товара нельзя.

Категории образуют отдельный справочник. Товар хранит идентификатор категории, а её название получает по связи с записью справочника. Из-за этого переименование раздела не требует исправлять каждый товар. В принятой модели у позиции одна категория; дерева разделов и нескольких категорий для одного товара нет. Для справочника предусмотрены собственные название и необязательное описание.

Цена и количество характеризуют разные стороны позиции. Цена вводится в рублях с точностью до копейки, количество — целым числом единиц. Нулевое количество означает отсутствие товара в наличии, но не удаляет его из каталога. Запись остаётся доступной для просмотра и редактирования. Хранящееся число является текущим остатком: отдельные документы поступления, списания и резервирования в модели отсутствуют.

Хранимую запись и входящий запрос описывают разные классы. Модель SQLAlchemy связывает поля с таблицей, схема Pydantic проверяет переданные значения, а словарь формы содержит строки, полученные из HTML. В Think Python классы и словари рассматриваются как разные способы организации данных [3, гл. 10, 14]. Здесь различие видно на цене: форма содержит строку рублей, схема разбирает её, модель получает целое число копеек.

Менеджеру нужны два основных способа обращения к данным. Из списка он ищет позицию, смотрит цену и наличие. В карточке получает описание и даты, затем при необходимости открывает редактирование. Категории изменяются на отдельных страницах. Все эти действия выполняются через браузер, который отправляет обычные HTTP-запросы к локальному приложению.

Количество физических единиц не совпадает с числом записей. Остаток 10 у синей ручки означает одну позицию и десять единиц товара. Чёрная ручка с другим артикулом увеличит число позиций на одну независимо от своего остатка. Это различие используется в сводных показателях: число товаров в наличии подсчитывается по записям, а общее количество — сложением их остатков.

Для имён товаров и категорий приняты разные правила повторов. Одинаковое название товара разрешено: позиции различаются по артикулу. У категории совпадение нормализованного названия запрещено. Так, «Бумага» и «БУМАГА» не создают два раздела. Само введённое написание сохраняется для вывода, а его служебная копия используется при сравнении.

Обязательные поля товара — название, артикул, категория, цена и количество. Описание можно оставить пустым; даты назначает приложение. Название категории ограничено 100 символами, товара — 200, оба описания — 2000. Для артикула разрешены латинские буквы, цифры, дефис и подчёркивание, его длина не превышает 64 символов. Эти ограничения можно проследить от HTML-полей до входных схем и моделей хранения.

1.2 ПОДХОДЫ К АВТОМАТИЗАЦИИ И ТРЕБОВАНИЯ К МОДУЛЮ

Задачу ведения ассортимента можно решать электронной таблицей, настройкой готовой системы или отдельным модулем. В сравнении рассматриваются поля товара, связь со справочником, проверка записи и способ выполнения операций. Коммерческие продукты в ходе работы не устанавливались и не испытывались. Поэтому таблица 1 описывает различия в организации данных и программы, а не рейтинг готовых решений.

В электронной таблице перечень позиций удобно представить строками. Однако проверку повторного артикула и согласование категории с отдельным справочником потребуется настраивать. Готовая система уже содержит свои сущности, формы и дополнительные процессы. Их состав нужно сопоставлять с заданием. Отдельный модуль позволяет ограничить реализацию товарами и категориями и явно задать правила записи, которые далее проверяются запросами.

Реализация построена как веб-приложение с небольшой моделью данных. По исходникам можно отдельно разобрать чтение запроса, его проверку и изменение таблицы. Такой размер программы подходит для сопоставления правила и выполняющего его кода. Работа с небольшими запускаемыми примерами рассмотрена в учебных материалах Г. Уилсона [7]. Сравнение способов организации каталога приведено в таблице 1.

Таблица 1 — Проектное сравнение подходов

Подход	**Что организуется**	**Соответствие задаче**
Таблица товаров	Столбцы свойств, правила ввода и связи нужно настроить	Подходит для исходного перечня; HTTP-модуль не реализует
Готовая система	Выбор продукта, настройка или расширение	Требует выбора продукта и настройки под задачу
Отдельный модуль	Собственные схема, операции, формы и проверки	Содержит операции каталога на Python и FastAPI

Бизнес-требования задают назначение каталога: хранить ассортимент и поддерживать актуальные цены и количество. Пользовательские требования описывают доступные менеджеру действия. Функциональные требования определяют ответ системы на эти действия. Например, после исправления цены она должна проверить ввод, сохранить выбранную запись и показать результат. Уникальный артикул и неотрицательное количество относятся к правилам данных и действуют независимо от того, отправлена форма или JSON-запрос.

Состав функций определяется заданием и принятой моделью. Нужны добавление, список, карточка, редактирование и удаление товаров, а также ведение категорий. Поиск по названию сочетается с категорией, границами цены и наличием. Название и цена сортируются в обоих направлениях; ещё один порядок выводит недавно изменённые записи первыми. Список разбит на страницы по десять товаров. Запрос ограничивает длину поиска 200 символами, принимает положительный номер страницы и отклоняет неизвестный способ сортировки.

Запись должна ссылаться на существующую категорию и иметь допустимые числовые значения. Повтор артикула товара и нормализованного имени категории запрещён. Сохранённые данные остаются в файле после остановки сервера. При удалении справочника учитываются связи: категория, к

которой относятся товары, сохраняется. Набор этих требований проверяется и через обработчики запросов, и отдельно на уровне таблиц.

Для операции заранее задаются допустимый случай и причины отказа. Например, создание товара проверяется с новым артикулом, с уже занятым артикулом и с неверной ценой. Для редактирования дополнительно требуется сравнить запись до и после отклонённого запроса. Проверка наблюдаемого результата относительно ожидаемого поведения и ограниченность отдельных примеров рассматриваются во вводной главе The Fuzzing Book [10].

Одно правило может потребовать нескольких разных проверок. При дубликате артикула API возвращает 409, тогда как прямая запись в таблицу должна встретить ограничение UNIQUE. Эти результаты проверяются раздельно. Если оставить только проверку API, отсутствие ограничения в базе может остаться незамеченным. Если проверить только таблицу, это ничего не скажет о коде и содержании HTTP-ответа.

Сценарий приёмки включает начальные данные, запрос и последующее чтение результата. Для запрета удаления сначала создаются категория и связанный товар. Затем отправляется запрос удаления и проверяется сохранность обеих записей. При редактировании читается тот же id и сравниваются его поля. Такой сценарий отделяет ожидаемый отказ от ошибки, при которой сервер сообщил о проблеме, но успел изменить часть данных.

У интерфейса проверяются свойства, необходимые для завершения действия: подписи полей, объяснение ошибки, переход к результату и подтверждение удаления. Отправленные значения должны оставаться доступными для исправления. Условия поиска сохраняются при переходах по списку. Это проверяемые требования к поведению страниц; оценка удобства работы по результатам опроса пользователей в работу не входит.

1.3 ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ

Для запуска требуется Python 3.12 и зависимости из файлов requirements.txt и requirements-dev.txt. Пакеты устанавливаются в отдельное

окружение. Команды получения исходников, установки и запуска приведены в README. Фиксация среды нужна для повторения описанных проверок; сохранение сведений о среде и происхождении результатов рассмотрено в Research Software Engineering with Python [4].

HTTP-обработчики написаны на FastAPI. Они возвращают JSON либо передают данные шаблону Jinja2, который формирует готовую HTML-страницу [13]. Отдельная сборка клиентского приложения не нужна. Шаблоны находятся в app/templates, стили и скрипт дат — в app/static; эти файлы выдаёт тот же сервер. Страница документации /docs использует Swagger UI из CDN, но для работы форм каталога открывать её не требуется.

SQLAlchemy используется для таблиц, запросов и сессии, Pydantic — для проверки входящих полей. Сессия выполняет commit и rollback и хранит состояние загруженных объектов [14]. Строгие числовые поля Pydantic нужны на границе API [16]: идентификатор категории и количество принимаются целыми JSON-числами. HTML-форма передаёт строки, поэтому для неё предусмотрен отдельный разбор до вызова общей схемы.

Каталог хранится в SQLite. Это файл, который открывается из процесса приложения без отдельного сервера СУБД [11]. На таком же движке создаются временные тестовые базы. Документация PostgreSQL из задания используется при разборе таблиц и ограничений [12]. Наличие ссылки на неё не означает, что текущая программа запускается на PostgreSQL: её подключение и ограничения рассчитаны на SQLite.

Зависимость от SQLite видна в исходниках: используются команды PRAGMA, проверка типа через typeof и проверка артикула через GLOB. Перенос на другую СУБД потребует пересмотреть эти выражения, настройки соединения и создание схемы. Нынешние результаты испытаний относятся к файловой базе SQLite. Их нельзя использовать как подтверждение работы с иной СУБД или при одновременном редактировании несколькими менеджерами.

Запросы к API и HTML выполняются тестовым HTTP-клиентом внутри pytest. Фикстуры готовят независимые данные для каждого случая [17]. Для действий с отображаемыми страницами применяется Playwright [19]. Сервер запускается через Uvicorn; команда указывает Python-модуль приложения, адрес 127.0.0.1 и порт 8000 [20]. Эти средства проверяют разные этапы работы одного модуля.

Основной интерфейс не рассчитывает на выполнение JavaScript. Поиск отправляет GET-запрос, сохранение формы — POST-запрос, ссылки открывают следующую страницу на сервере. Список, итоговые показатели и диаграммы формируются из полученных данных при подготовке HTML. Локальный скрипт меняет только представление дат: значение UTC выводится во времени браузера.

Соединение SQLite создаётся функцией create_sqlite_engine. Параметр check_same_thread=False позволяет использовать его в принятой схеме обработки запросов. При каждом новом подключении обработчик события connect включает PRAGMA foreign_keys=ON. Сессия при этом не является общей для всех обращений: она создаётся зависимостью запроса. Функция init_db получает движок параметром, поэтому одну схему можно подготовить в рабочем файле и во временном файле теста.

Порядок развёртывания можно проверить без ручной подготовки таблиц: получить проект, создать окружение, установить зависимости и запустить приложение. При старте создаются недостающие таблицы. Повторный запуск не удаляет записи и не переносит старую схему на новую автоматически. Файл data/catalog.db исключён из Git; после получения исходников каталог заполняется своими записями либо демонстрационным загрузчиком. Команда pip check проверяет зависимости, а pytest — поведение программы.

2 ПРОЕКТИРОВАНИЕ МОДУЛЯ УПРАВЛЕНИЯ КАТАЛОГОМ

2.1 ПОЛЬЗОВАТЕЛЬСКИЕ СЦЕНАРИИ И ФУНКЦИОНАЛЬНАЯ МОДЕЛЬ

Работа с пустым каталогом начинается со справочника. Менеджер вводит название категории и, при необходимости, описание. Созданная категория появляется среди значений формы товара. Если в справочнике ещё нет записей, интерфейс предлагает сначала заполнить его. Повторное имя проверяется после нормализации, поэтому изменить регистр букв для создания второго такого же раздела нельзя.

При добавлении товара обязательны название, артикул, категория, цена и количество. После успешной отправки формы открывается новая карточка. Редактирование начинается с той же карточки, но форма уже содержит сохранённые значения. Менеджер меняет нужные поля и отправляет полный набор данных. Идентификатор товара не меняется. При ошибке сервер возвращает отправленную форму, чтобы можно было исправить ввод без повторного заполнения остальных полей.

Список позволяет задать несколько условий сразу. Слово для поиска, выбранная категория, границы цены и наличие действуют совместно; незаполненный фильтр не добавляет ограничения. Пустой результат после такого отбора отличается от отсутствия товаров в каталоге. В первом случае полезно изменить условия, во втором — создать запись. Основные требования, способы их реализации и проверяемые результаты сопоставлены в таблице 2.

Таблица 2 — Связь требований с проверками

Требование	**Реализация**	**Признак корректности**
CRUD товара	Форма, карточка, API	Изменения видны после чтения; ID сохраняется
Категории	Справочник и связь	Повторное имя и удаление занятой категории отклоняются
Поиск	Нормализованное имя	Русский регистр не меняет результат
Фильтры	Условия общего запроса	Категория, цена и наличие применяются вместе
Пагинация	LIMIT 10 и OFFSET	120 страниц для 1200 товаров
Неправильный ввод	Схемы и форма	Статус 422; заполненные значения сохранены
Данные после старта	Файловая SQLite	Повторное открытие не удаляет записи

Удаление разделено на просмотр и подтверждение. GET-запрос открывает страницу с записью, а POST-запрос выполняет изменение базы. Отмена возвращает к товару. При попытке удалить занятую категорию вместо подтверждения показывается причина отказа и переход к её товарам. Сам POST-обработчик также проверяет связь: отсутствие кнопки на странице не заменяет серверное правило.

По HTTP-коду можно различить причины неуспешного действия. Неверное значение даёт 422, отсутствующая запись — 404, повтор артикула или занятая категория — 409. В API вместе с кодом возвращается JSON-описание, в браузере — HTML-страница. Успешное создание через API имеет код 201, удаление — 204. Для сохранённой веб-формы используется 303 с адресом страницы результата.

В функциональном тесте проверяется наблюдаемое действие пользователя; тест отдельного компонента помогает точнее локализовать отказ

[6, гл. 4]. Для этого каталога уровни разделены явно. Проверка таблицы выполняет операции хранения, тест HTTP-клиента отправляет API- или HTML-запрос, браузерный сценарий заполняет форму и нажимает ссылку либо кнопку. Один уровень не заменяет остальные: верный ответ функции ещё не подтверждает правильный переход в браузере.

Во время редактирования существуют сохранённая запись и новый ввод. До commit они не должны считаться одним состоянием. При отклонённом запросе форма показывает отправленные строки, а повторное чтение через API — прежние поля товара. После принятого изменения карточка загружает уже сохранённые значения. Это различие определяет, какие данные нужно сравнивать в тесте успешного и неуспешного сохранения.

Для обращения к записи используется её id, а не номер строки в выдаче. После сортировки или изменения фильтра положение товара может стать другим, но ссылка на карточку останется той же. Если товар уже удалён отдельным запросом, чтение по его идентификатору возвращает 404. Обработчик не подменяет отсутствующую запись соседним товаром из списка.

Проверка адреса предшествует поиску записи. Отрицательный или слишком большой id отклоняется с 422; положительный id без соответствующего товара приводит к 404. Для списка различаются ещё два случая: несуществующая категория и существующая категория без результатов. В первом случае функция каталога сообщает об отсутствии справочной записи. Во втором запрос корректен и возвращает пустой список. Ошибочный ценовой диапазон с нижней границей выше верхней отклоняется до выборки.

Ответ после создания товара нужен для продолжения сценария. Из него можно получить id и выполнить чтение, изменение, а затем удаление этой же позиции. Вместе с category_id возвращается название категории, цена представлена строкой рублей. Тесты используют эти поля для проверки связи и значений между запросами. Так устанавливается результат всей последовательности действий с записью, а не только первого ответа.

2.2 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ И ОГРАНИЧЕНИЙ ЦЕЛОСТНОСТИ

Схема базы содержит две сущности. Category хранит идентификатор, название, его нормализованную форму и описание. Product содержит собственный ключ, артикул, ссылку на категорию и характеристики позиции. Между ними установлена связь один ко многим. На рисунке 1 показаны сущности и направление ссылки, а в таблице 3 перечислены представления полей и ограничения.

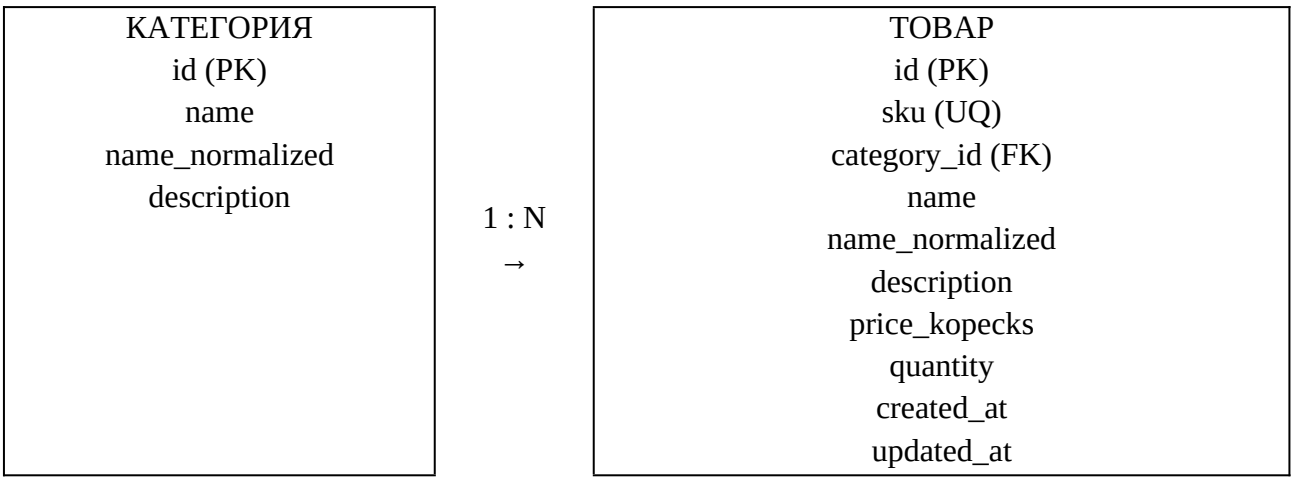


Рисунок 1 — Логическая структура данных каталога

Источник: составлено по модели и исходному коду проекта.

В таблице products поле category_id ссылается на categories.id. Это внешний ключ с правилом ON DELETE RESTRICT. Если на категорию ссылается хотя бы один товар, удаление её строки отклоняется. Первичные и внешние ключи рассматриваются в документации PostgreSQL [12]; в действующей схеме их создаёт SQLAlchemy для SQLite. Проверка наличия категории в функции приложения дополняет это ограничение и даёт понятное описание ошибки.

Для денег используется поле price_kopecks целого типа. Например, 49,90 рубля сохраняются как 4990. Входная строка разбирается через Decimal после проверки формата: допустимы точка и запятая и не более двух дробных знаков. Третий знак вызывает ошибку, а не округление. Документация decimal

описывает точное представление десятичных величин [11]. В ответе количество копеек преобразуется обратно в рубли с двумя знаками после разделителя.

Количество ограничено неотрицательными целыми числами. В схеме запроса проверяется сам тип, в SQLite — выражение CHECK с typeof и сравнением с нулём. Для артикула товара и нормализованного названия категории заданы уникальные поля. Предварительный поиск повтора позволяет подготовить сообщение до записи, но окончательное правило действует в таблице. Оно остаётся значимым и тогда, когда два близких по времени запроса прошли предварительную проверку.

Сравнение названий выполняется функцией normalize_name. Она применяет NFKC, удаляет пробелы по краям и вызывает casefold. Преобразование NFKC относится к нормализации Unicode [11]; оно не является нормализацией реляционных таблиц. В базе остаются две строки: исходное название для отображения и name_normalized для поиска. Введённый поисковый фрагмент проходит ту же обработку. Для русского текста это снимает зависимость результата от регистра, но не задаёт отдельного словарного порядка сортировки.

Таблица 3 — Основные поля модели данных

Сущность и поле	**Представление**	**Правило**
Category.id	Целое	Первичный ключ
Category.name	Строка до 100 символов	Непустое название
Category.name_normalized	Нормализованный текст	Уникальность без учёта регистра; служебное поле
Category.description	Текст или NULL	Не более 2000 символов
Product.id	Целое	Первичный ключ
Product.name	Строка до 200 символов	Непустое название

Сущность и поле	Представление	Правило
Product.name_normalized	Нормализованный текст	Поиск; заполняется вместе с названием
Product.sku	Строка до 64 символов	Уникальный; латинские буквы, цифры, дефис, подчёркивание
Product.category_id	Целое	Внешний ключ; категория существует; RESTRICT
Product.price_kopecks	Целые копейки	От 0 до 2^63 − 1; на входе рубли строкой
Product.quantity	Целое количество	От 0 до 2^63 − 1
Product.description	Текст или NULL	Не более 2000 символов
Product.created_at	Дата и время UTC	Назначается при создании
Product.updated_at	Дата и время UTC	Назначается при создании и изменении

Служебная копия названия не включается в публичные схемы CategoryRead и ProductRead. Клиент получает обычное name, а не обе версии строки. Значение name_normalized заполняется при присваивании названия модели. Такое разделение позволяет сохранить введённое написание в карточке и применять одинаковые правила при поиске и проверке повтора категории.

Для необязательного описания пустая строка после удаления крайних пробелов превращается в NULL. Даты назначаются приложением по соглашению UTC. SQLite хранит DateTime без встроенного часового пояса, поэтому при подготовке ответа API обозначение UTC добавляется явно. В HTML дата передаётся элементу time. Изменяется только её видимое местное представление, а не сохранённое значение.

Ограничения таблицы отвечают за разные ошибки. NOT NULL не допускает отсутствие обязательного значения, CHECK проверяет выражение, UNIQUE — повтор, внешний ключ — наличие связанной категории.

Положительный category_id может удовлетворять типу и диапазону и при этом не обозначать ни одной категории. Здесь нужна именно проверка связи. Тесты с прямой записью в базу позволяют рассмотреть каждое ограничение отдельно от входной схемы API.

Верхняя граница целого SQLite составляет 2^63 − 1. Для price_kopecks это число копеек, поэтому максимальная входная сумма рублей — 92233720368547758.07. Следующая копейка уже превышает предел. Количество ограничивается тем же целым числом без денежного пересчёта. Идентификаторы проверяются параметром Path до обращения к базе. Для page верхняя граница дополнительно делится на размер страницы: произведение номера на десять должно помещаться в диапазон при вычислении OFFSET.

Фильтр цены использует то же представление, что и хранимая запись. Граница 49,90 превращается в 4990, после чего запрос сравнивает целые значения. В сравнении не участвуют приближённые числа float. В публичной схеме нет входного поля price_kopecks: клиент передаёт price строкой рублей. Поэтому преобразование и проверка предела выполняются в одном месте до изменения модели.

Нормализованная строка обновляется валидатором модели вместе с названием. Если исправить только name прямым SQL-запросом, Python-валидатор не выполнится и служебная копия может остаться прежней. Тогда отображаемое название и поиск перестанут соответствовать друг другу. Операции каталога присваивают поля через модели. При разработке другого способа импорта нужно будет сохранить и это правило обновления, а не ограничиться записью столбца name.

2.3 АРХИТЕКТУРА ПРИЛОЖЕНИЯ И ПРОЕКТИРОВАНИЕ ИНТЕРФЕЙСА

Файлы приложения разделены по выполняемой работе. В database.py создаются движок и сессии, models.py описывает таблицы, schemas.py — допустимые запросы. Общие операции находятся в catalog.py. Маршруты API и

страниц получают входные значения и выбирают форму ответа. В шаблонах остаётся вывод, в статических файлах — оформление и перевод дат. Назначение компонентов приведено в таблице 4, прохождение запроса — на рисунке 2.

Таблица 4 — Ответственность компонентов

Компонент	**Назначение**
database.py	Движок, внешний ключ, сессия на запрос
models.py / schemas.py	Таблицы, ограничения и проверка входных данных
catalog.py	Общие операции, транзакции, поиск, фильтры и итоги
web.py	Форматирование, подготовка диаграмм, версии CSS и JavaScript
routers/api.py	JSON-контракт и статусы HTTP
routers/pages.py	HTML-формы, сообщения и перенаправления
templates / static	Представление, оформление и местное время
demo_data.py / seed_demo.py	Воспроизводимые примеры, backup и загрузка

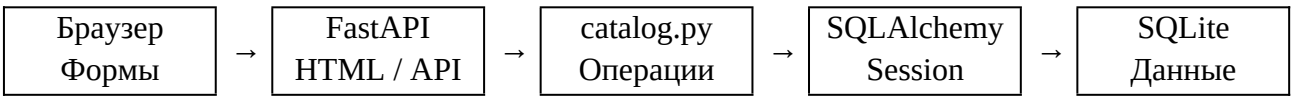


Рисунок 2 — Обработка запроса в приложении

Источник: составлено по модели и исходному коду проекта.

HTML- и JSON-маршруты обращаются к одним функциям catalog.py. Например, изменение товара не реализовано заново для каждой формы ответа. Разделение прикладной операции и кода взаимодействия описано в книге Architecture Patterns with Python [5, гл. 4]. Здесь оно выполнено функциями, принимающими сессию SQLAlchemy. Дополнительный репозиторий, очередь событий и отдельный объект Unit of Work в структуру не включены.

Изменение базы заканчивается явным commit. При IntegrityError функция выполняет rollback, после которого сессия снова может использоваться.

Транзакционные границы позволяют рассматривать изменение как законченную операцию; этот принцип связан с описанием Unit of Work [5, гл. 6]. Конкретное поведение сессии при фиксации и откате определяется SQLAlchemy [14]. Реализация ограничивается этими средствами без отдельного класса для транзакции.

Для списка, карточки, форм и подтверждения удаления предусмотрены отдельные шаблоны. Они наследуют общий базовый шаблон с меню товаров и категорий. На широком экране меню расположено слева; на узком оно становится верхней строкой. Длинная таблица прокручивается в собственном контейнере, а поля формы выстраиваются по одному. Белый фон и розовые акценты задаются локальным CSS.

Файл рабочей базы — data/catalog.db. Его путь вычисляется от проекта и не зависит от каталога, из которого вызван Uvicorn. Работа с сохранёнными объектами и базами рассматривается в материалах Г. Уилсона [7]. В данном приложении состояние каталога хранится в таблицах SQLite и читается через ORM-модели. Остановка HTTP-процесса не удаляет этот файл.

CatalogError переносит код и описание ожидаемого отказа из функции каталога в слой ответа. Для API обработчик возвращает JSON с detail, для страницы формируется HTML. Сам поиск занятого артикула от этого не меняется. Шаблону передаётся подготовленный результат: он может вывести сообщение, но не выполняет удаление или сохранение записи.

Сессию предоставляет зависимость get_db. Обработчик получает её при запросе, передаёт в функцию каталога и после завершения освобождает. В тесте та же зависимость подменяется сессией временной базы. Маршрут и его проверки остаются теми же, меняется только подключение к данным. Команды commit и rollback выполняются в каталоге; шаблон не открывает соединение и не определяет момент сохранения.

Состав страницы определяется её назначением. Списку нужны названия, категории, цены и количество; карточке также нужны описание и даты. Форма содержит редактируемые поля и ограничения длины. Эти HTML-ограничения

облегчают ввод, однако запрос всё равно проверяется сервером: его можно отправить напрямую. Значения полей для вывода и объекты модели поэтому подготавливаются отдельно.

3 РАЗРАБОТКА И ТЕСТИРОВАНИЕ МОДУЛЯ

3.1 РЕАЛИЗАЦИЯ ОПЕРАЦИЙ С ТОВАРАМИ И КАТЕГОРИЯМИ

Приложение создаётся в app/main.py. Обработчик lifespan вызывает подготовку таблиц при старте и освобождает движок при остановке. Путь data/catalog.db вычисляется от папки проекта. Команда Uvicorn с адресом 127.0.0.1 и портом 8000 запускает локальный HTTP-сервер [20]. Вызов create_all создаёт отсутствующие таблицы, но не меняет ранее созданную схему и не заменяет существующие данные.

У каждой сущности API имеет пять операций: список, создание, чтение по id, изменение и удаление. Всего их десять. Запрос товара проверяется схемой ProductWrite; ответ собирается по ProductRead. Для категории используются CategoryWrite и CategoryRead. В ответе товара есть category_name, хотя при записи клиент передаёт category_id. Внутренняя модель с name_normalized и price_kopecks целиком не сериализуется. Цена и даты подготавливаются в публичном формате.

Функция create_product проверяет существование категории и свободный артикул, создаёт Product и фиксирует запись. После commit вызывается refresh, чтобы получить сохранённое состояние. При обновлении сначала находится товар и проверяются новые ссылки и артикул, затем присваиваются поля. Поэтому обнаруженный заранее дубликат не успевает изменить цену. При нарушении ограничения во время commit выполняется rollback [14]. Отдельный тест после такого отказа сравнивает запись и пробует допустимое обновление.

Перед удалением категории функция delete_category ищет один связанный товар с LIMIT 1. Для решения о запрете полный список не нужен. Если товаров нет, категория удаляется и транзакция фиксируется. Удаление самого товара не затрагивает справочник. При обычном UPDATE значение updated_at назначается моделью; клиент не передаёт его в форме или схеме записи.

Успех веб-формы завершается кодом 303 и адресом карточки либо списка. Такой ответ соответствует назначению See Other в RFC 9110 [18, п. 15.4.4]. Последующий GET читает результат и отделяет его от изменяющего POST. При неверном вводе или конфликте перенаправления нет: обработчик формирует страницу с соответствующим кодом 422 либо 409. Различие проверяется и по HTTP-ответу, и по состоянию записи.

HTML не передаёт целые числа как отдельный тип. Адаптер integer_fields копирует полученный словарь и разбирает category_id и quantity, если после удаления крайних пробелов в них от одной до девятнадцати цифр. Например, строка «20» становится числом 20, а «20.0» остаётся неподходящим значением. Затем действует строгая схема Pydantic [16]. Исходные строки не теряются и могут быть возвращены форме при ошибке.

Функция price_to_kopecks проверяет цену в несколько шагов. Сначала удаляются крайние пробелы, ограничивается длина строки и проверяется регулярное выражение. Допускаются цифры и точка либо запятая с одной или двумя дробными цифрами. Затем создаётся Decimal, проверяется верхняя граница и вычисляются копейки. Значения «49», «49.9» и «49,90» допустимы; экспоненциальная запись, отрицательное число и NaN не проходят проверку формата.

В JSON количество и категория проходят проверку strict=True. Она различает целое число 1, строку «1», дробную запись 1.0 и логическое true. Для двух этих полей принимается только первый вариант. Это правило не меняется из-за того, что веб-форма имеет собственный разбор строк: после адаптера форма вызывает ту же ProductWrite, что и API.

InputModel запрещает поля, не описанные во входной схеме. Поэтому передать id или price_kopecks вместе с обычными полями товара нельзя. PUT используется для полного набора редактируемых значений: одного нового name недостаточно. При поиске занятого артикула текущий id исключается из сравнения. Благодаря этому неизменённый артикул своего товара не считается повтором, а артикул другой позиции вызывает конфликт.

В _commit перехватывается IntegrityError. Сначала восстанавливается сессия через rollback, затем разбирается сообщение SQLite. Известны три причины: повтор имени категории, занятый артикул и нарушение внешнего ключа. Для них формируется CatalogError с кодом 409. Неизвестное нарушение передаётся дальше как исходное исключение. Функция не подменяет любую ошибку базы сообщением о дубликате. Для связи описание можно уточнить: при удалении речь идёт о товарах категории, при записи товара — об изменившейся связанной категории.

3.2 РЕАЛИЗАЦИЯ ПОИСКА, ФИЛЬТРАЦИИ И ВЕБ-ИНТЕРФЕЙСА

Условия списка собирает _product_conditions. Поиск применяется к name_normalized через contains с autoescape=True. Процент, подчёркивание и служебная косая черта рассматриваются как введённые символы, а не как команды для LIKE. Поиск передаётся параметром. Условия категории, цены и положительного количества добавляются к этому же запросу. Порядок сортировки выбирается из готовых выражений SQLAlchemy; произвольная строка SQL из параметра sort не строится.

Доступны name_asc, name_desc, price_asc, price_desc и updated_desc. Эти значения описаны как Literal во входной схеме и связаны с выражениями order_by в catalog.py. После выбранного поля всегда добавляется Product.id по возрастанию. Для нескольких товаров с одинаковой ценой это даёт определённый дополнительный порядок. Случайный порядок равных значений мог бы мешать проверке состава соседних страниц.

List_products возвращает пару: записи выбранной страницы и общее количество найденных товаров. Число результатов определяется отдельным COUNT с теми же условиями. Вторая выборка использует LIMIT 10 и OFFSET. Пустой результат при допустимом запросе не считается ошибкой. Количество страниц вычисляется с округлением вверх, а при отсутствии товаров интерфейс сохраняет первую страницу.

Шаблон получает товары и значения фильтров через контекст Jinja2 [13]. Пользовательские строки не помечаются safe, поэтому HTML-символы в названии или описании выводятся как текст. Возможности автоматического экранирования и его отключения описаны в документации Jinja [15]. В проверках используются строки с тегами. Это позволяет установить, что название не превращается в исполняемую разметку, в том числе в подписях диаграммы.

Учебный набор строится из 12 категорий, десяти типов товара в каждой и десяти вариантов типа. Получается 1200 записей и по 100 в каждой категории. Артикулы имеют постоянный вид DEMO-01-01-01, поэтому повторная загрузка находит те же позиции. В этом наборе 133 нулевых остатка, цены от 19,00 до 346,10 рубля. Данные описывают синтетический ассортимент для проверки списков, а не действующий прайс-лист поставщика.

Загрузчик сначала создаёт резервную копию существующего файла методом SQLite backup [11]. Только после этого начинается транзакция добавления примеров. До создания категории вычисляются недостающие артикулы. Если все её демонстрационные товары уже есть, новый раздел не создаётся. При найденном нормализованном имени используется существующая категория и её описание. Занятые артикулы пропускаются, поэтому повторный запуск не перезаписывает исправленные название, цену и принадлежность товара.

Для совместного отбора в тесте используются синяя ручка за 49,90 рубля, гелевая за 100,00 и красная за 99,00 с нулевым остатком. Запрос «РУЧКА» с категорией, границами 49,90–100,00 и наличием возвращает первые две позиции. Здесь одновременно проверяются русский регистр, равенство цены границе диапазона и исключение по количеству. Удовлетворить только части условий недостаточно.

Номер страницы задаёт смещение (page − 1) × 10. При 13 результатах первая страница содержит десять записей, вторая — три. Номер после последней страницы оставляет список пустым; интерфейс не выдаёт под этим

номером другие товары. Ссылка «Назад» в таком случае возвращает на последнюю существующую страницу. В обычном списке показаны первая, последняя и до пяти соседних страниц с многоточиями между удалёнными номерами.

Для каждой строки списка требуется название категории. Поэтому товары выбираются с joinedload(Product.category), и связанные сведения доступны шаблону вместе с результатом запроса. При обращении к product.category.name не нужен отдельный запрос для каждой строки. Тест проверяет соответствие названий категорий товарам. Сравнение производительности этой загрузки с другими вариантами не проводилось.

Поисковая форма использует GET, поэтому её значения остаются в адресе. При формировании ссылок urlencode сохраняет q, category_id, цены, наличие и сортировку; меняется только page. Новая отправка формы не содержит старого номера и начинает выдачу с первой страницы. Адрес воспроизводит условия отбора, но не фиксирует снимок данных: после редактирования товара результаты по нему могут измениться.

Под списком показаны четыре итога: позиции в наличии, позиции с нулевым остатком, сумма единиц и стоимость остатка. Их готовит summarize_products с теми же условиями, что использованы для списка. LIMIT и OFFSET к этому расчёту не добавляются. Поэтому переход со страницы 1 на страницу 2 не изменяет итоги, а изменение фильтра пересчитывает их по новой выборке.

Для расчёта читаются только цена в копейках, количество и название категории. У каждой записи определяется признак положительного остатка, quantity добавляется к числу единиц, а price_kopecks × quantity — к стоимости. Суммы накапливаются целыми числами Python. Это сохраняет точность копеек даже тогда, когда несколько допустимых значений в сумме превышают предел одного INTEGER SQLite. Весь объект товара для такого прохода не загружается.

Разница между показателями видна на тестовых данных. У синей ручки цена 49,90 рубля и количество 20, у гелевой — 100,00 и 3, у красной — 99,00 и 0, у тетради — 20,00 и 4. Получается три позиции в наличии и одна без остатка, 27 единиц и 1378,00 рубля стоимости. Это сумма 998,00, 300,00, 0 и 80,00 рубля. Тот же набор используется в HTML-тесте, поэтому пример можно проверить по реализации.

После отбора ручек с наличием и границами 49,90–100,00 остаются две позиции, 23 единицы и 1298,00 рубля. Если выбрать только бумажную категорию, итог относится к тетради. Эти случаи проверяют, что сводка действительно использует фильтры списка. В отдельном тесте двенадцать товаров распределены по двум страницам, но на обеих страницах выводятся одинаковые итоги по всем двенадцати записям.

Первая диаграмма показывает стоимость остатка по категориям. Суммы относятся к текущим ценам каталога. Если категорий больше шести, отдельно выводятся пять с наибольшей стоимостью, а остальные складываются в одну полосу с указанием их числа. Сумма всех полос должна совпадать со стоимостью в сводке. Равные суммы упорядочиваются по названию. Для пустой выборки или нулевой стоимости выводится пояснение вместо полос.

Кольцевая диаграмма использует число позиций, а не сумму единиц. Процент получается делением количества записей с положительным quantity на количество найденных записей и округляется до десятых. Поэтому один товар с большим остатком не заменяет несколько позиций в этом показателе. При пустой выдаче доля не вычисляется. Подписи и числа формируются на сервере; диаграммы продолжают работать при отключённом JavaScript.

Подготовка чисел и внешний вид диаграмм разделены. CatalogSummary содержит итоги и суммы категорий, catalog_charts готовит долю и ширину полос, шаблон выводит HTML и SVG. Исходное название категории экранируется так же, как в карточке. Для денежных итогов форматирование добавляет разделители тысяч и две дробные цифры, не переводя сумму в float. Снимок сводки и диаграмм приведён на рисунке А.5.

В ссылку на CSS и скрипт включён короткий хеш содержимого файла. При изменении стилей меняется адрес ресурса, и браузер загружает новую версию. Для неизменённого файла адрес остаётся постоянным. Тест проверяет обновление ссылки даже при сохранённой дате изменения файла. Эта мера устраняет ситуацию, когда новая HTML-страница открывается со старым оформлением из кеша.

3.3 ТЕСТИРОВАНИЕ, АНАЛИЗ РЕЗУЛЬТАТОВ И ОГРАНИЧЕНИЯ РЕШЕНИЯ

Для новой редакции записки проверки повторены 12.11.2026 в Linux на Python 3.12. Каждый тест получает временный файл SQLite, схему и сессию. В HTTP-клиенте подменяется get_db, поэтому обращения выполняются к тестовым данным. Подготовка и освобождение таких ресурсов описаны в документации pytest [17]. Протокол сохранён в checks/tests.xml; рабочий data/catalog.db для испытаний не использовался.

Пройдены 173 автоматических теста: 35 для базы данных, 81 для API, 50 для HTML и 7 для загрузчика. В числе HTML-проверок теперь есть номера страниц, расчёт итогов, диаграммы и обновление ссылки на CSS. Прежние протоколы содержат 154 теста; увеличение связано с 19 дополнительными случаями интерфейса. Параметризованные наборы pytest считаются отдельно, поэтому число пройденных случаев больше числа функций test_. Таблица 5 содержит результаты текущего прогона.

В прогоне осталось одно предупреждение Starlette о применении HTTPX в TestClient. Проваленных тестов не было. Это зафиксированное состояние зависимостей, а не результат измерения надёжности приложения. Повторение набора после изменений проверяет сохранение уже описанного поведения; роль таких проверок при сопровождении рассмотрена в Software Engineering at Google [8, гл. 11].

Основной браузерный прогон 12.11.2026 прошёл 14 сценариев в Chromium 154.0.8037.57. Они включают создание категории и товара,

изменение цены и количества, ошибочную цену, составной поиск, переходы страниц и удаление. Отмена и подтверждение удаления проверяются раздельно. После действия данные читаются через API. Шесть случаев записаны в отчёте подробно и входят в число четырнадцати. Сценарий использует синхронный интерфейс Playwright [19].

Для широкого окна использован размер 1440 × 1000, для мобильного — 390 × 844. В проверке большого каталога добавлена ширина 320 пикселей. Сравнивались ширина документа и окна, расположение номеров страниц и карточек диаграмм. Переполнения страницы не зарегистрировано; прокрутка таблицы остаётся внутри её контейнера. Воспроизведение условий полезно для поиска причины отказа, как отмечено в The Debugging Book [9]. По одним снимкам без условий запуска такой вывод сделать нельзя.

На другом временном файле с 1200 товарами выполнены 13 дополнительных браузерных проверок. Проверены номера в начале, середине и конце списка, переход по номеру, совместные фильтры, пустой поиск, неверные границы цены и работа без JavaScript. Отдельные случаи проверяют две диаграммы и загрузку свежего CSS. Для полного набора получены 1067 позиций в наличии, 133 без остатка, 80 654 единицы и 13 233 058,70 рубля стоимости остатка. Эти итоги независимо пересчитаны по всем страницам API через Decimal.

Таблица 5 — Фактические результаты проверки

Набор	**Количество**	**Результат**
База данных	35 тестов	Пройдены
JSON API	81 тест	Пройдены
HTML-страницы	50 тестов	Пройдены
Загрузчик примеров	7 тестов	Пройдены
Браузер, основные сценарии	14 сценариев	Пройдены в Chromium
Браузер, выборка и диаграммы	13 проверок	Пройдены в Chromium
SQLite большого набора	integrity_check, foreign_key_check	ok; ошибок связей нет

В приложении приведены снимки текущего интерфейса от 12.11.2026: каталог, форма, ошибочная цена, запрет удаления категории и диаграммы. Они получены на временных базах, данные которых перечислены в протоколах. Исторические отчёты от 21.10 и 09.11 сохранены отдельно. Разделение исходных условий и последующих изменений соответствует документированию происхождения результатов [4]. Проверка сохранения файла между запусками входит в автоматические тесты.

Результаты относятся к локальной работе приложения с SQLite. Проверки не включают нагрузку, одновременную запись нескольких менеджеров, PostgreSQL и другие браузеры. Авторизации, журнала изменений и миграций в программе нет. Для этих режимов потребуются изменения и отдельные испытания. Числа в сводке каталога характеризуют сохранённые товары; операции заказов и складские документы по ним не выполняются.

Уровни проверки помогают отличить ошибку данных от ошибки ответа. Прямой INSERT в тесте базы обращается к CHECK или внешнему ключу. Запрос к API проходит входную схему, функцию каталога и подготовку результата. HTML-тест дополнительно читает поля и ссылки страницы. Например, неверная цена должна сохранить прежнюю запись, а форма —

вернуть введённую строку. Сравнение только статуса 422 не охватывает оба требования.

Набор граничных значений вытекает из схем. Название проверяется на пустоту, пробелы и превышение длины. Цена — на ноль, лишние дробные цифры, экспоненциальную запись и верхнюю границу SQLite. Для количества отдельно нужны дробное, отрицательное, логическое и слишком большое значения. Поиск проверяется русскими прописными буквами и буквальным вводом %, _ и /. Такие примеры адресованы конкретным операциям преобразования и сравнения.

В проверке занятого артикула сначала читается исходный товар. После конфликтного обновления его поля сравниваются с сохранённым образцом, затем отправляется допустимое изменение. Это проверяет и отсутствие частичного сохранения, и продолжение работы после ошибки. Другой случай переносит товар в новую категорию: ответ должен показать её id и название. Прежняя категория после переноса может быть удалена, если стала пустой.

Для отката загрузки тест вызывает исключение во время INSERT товара. После него в базе не должно остаться добавленных этой транзакцией товаров и категорий. Ошибка создания backup проверяется отдельно: запись набора не должна начинаться. Ещё один случай использует WAL: данные фиксируются в открытой базе, затем проверяется их наличие в резервной копии. Эти проверки обращаются к разным границам операции, а не повторяют один успешный запуск.

Повторное наполнение проверяется на базе с собственными записями. После первого добавления набора меняются название и цена одной демонстрационной позиции. Второй запуск должен сохранить эти значения, исходное описание категории и пользовательский товар. Дополнительно сравниваются число добавленных и пропущенных записей и отсутствие повторных артикулов. Один только общий счётчик не обнаружил бы замену значений внутри существующей строки.

Совпадение с демонстрационным артикулом рассматривается как занятая позиция независимо от её происхождения. Тест заранее создаёт собственный DEMO-01-01-01 в другой категории. Загрузчик добавляет 1199 товаров и пропускает один; у исходной записи остаются прежние название и категория. Проверка показывает, почему решение о пропуске принимается по артикулу, а не по тексту описания.

Для проверки уже запущенного обычного сервера предусмотрен scripts.check_ready. Команда читает страницы, формы, локальные CSS и JavaScript, ответ health и списки API. По OpenAPI проверяются десять операций каталога. Кроме кода 200 проверяется содержимое: нужный HTML-заголовок, форма, формат JSON и поля списка. Наличие ответа на порту само по себе не подтверждает, что запущен нужный модуль.

Та же команда через SQLAlchemy читает таблицы, столбцы товара, внешний ключ и настройку foreign_keys. Запись товаров в неё не входит. Это дополняет /health: обработчик сообщает о доступности HTTP-процесса, но не выполняет запрос к каталогу. Поэтому отдельно сверяется структура базы. Обычную готовность сервера и результаты изолированных испытаний следует рассматривать раздельно.

К новой редакции приложены протокол pytest и два браузерных отчёта с датой 12.11.2026. В них указаны условия запуска и проверяемые действия. Их можно сопоставить с файлами tests/, scripts/browser_check.py и кодом списка. В репозитории остаются более ранние материалы, но текущие числа в таблице 5 взяты из нового прогона. Эти результаты подтверждают перечисленные случаи, а не отсутствие всех возможных ошибок программы.

ЗАКЛЮЧЕНИЕ

Разработанный модуль хранит товары и категории и предоставляет веб-интерфейс и JSON API. Использованы Python 3.12, FastAPI, SQLAlchemy и SQLite.

Определены обязательные поля и правила записи, сопоставлены способы ведения каталога. Проверка запросов, операции хранения и подготовка ответа разделены между компонентами приложения.

Спроектирована связь одной категории с несколькими товарами. Уникальность артикула, числовые ограничения и внешний ключ проверяются до записи и в базе. Цена хранится в копейках; отклонённое изменение сохраняет прежние данные.

Реализованы десять операций API и действия в браузере. Поиск сочетается с категорией, ценой и наличием, переходы сохраняют фильтры. Сводка и диаграммы относятся ко всей найденной выборке.

Подготовлены 1200 товаров в 12 категориях. Загрузчик создаёт резервную копию, добавляет недостающие записи и сохраняет пользовательские изменения. Проверены повторный запуск, откат и содержимое копии.

В прогоне 12.11.2026 пройдены 173 автоматических теста, 14 основных и 13 дополнительных браузерных проверок Chromium. Протоколы и снимки сохранены вместе с условиями получения результатов.

Модуль предназначен для локального ведения каталога. Авторизация, заказы, складские движения и многопользовательский режим не реализованы. Ограничения рассмотрены в разделе 3.3, исходники и инструкция запуска доступны в репозитории.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19.201–78. Единая система программной документации. Техническое задание.

2. Московский финансово-юридический университет МФЮА. Методические указания по подготовке и оформлению курсовых работ/проектов. — Москва, 2025. — Текст : электронный.

3. Downey, A. B. Think Python: How to Think Like a Computer Scientist / A. B. Downey. — 3rd ed. — O’Reilly Media, 2024. — ISBN 978-1-098-15543-8. — URL: https://github.com/AllenDowney/ThinkPython/tree/v3 (дата обращения: 21.10.2026). — Текст : электронный.

4. Research Software Engineering with Python: Building Software that Makes Research Possible / D. Irving, K. Hertweck, L. Johnston [et al.]. — Chapman & Hall/CRC Press, 2021. — ISBN 978-0-367-69834-8. — URL: https://github.com/merely-useful/py-rse (дата обращения: 21.10.2026). — Текст : электронный.

5. Percival, H. Architecture Patterns with Python / H. Percival, B. Gregory. — O’Reilly Media, 2020. — ISBN 978-1-492-05220-3. — URL: https://github.com/cosmicpython/book (дата обращения: 21.10.2026). — Текст : электронный.

6. Percival, H. J. W. Test-Driven Development with Python / H. J. W. Percival. — 3rd ed. — O’Reilly Media, 2025. — ISBN 978-1-098-14871-3. — URL: https://github.com/hjwp/Book-TDD-Web-Dev-Python (дата обращения: 21.10.2026). — Текст : электронный.

7. Wilson, G. Software Design by Example: A Tool-Based Introduction with Python / G. Wilson. — URL: https://github.com/gvwilson/sdxpy (дата обращения: 21.10.2026). — Текст : электронный.

8. Winters, T. Software Engineering at Google: Lessons Learned from Programming Over Time / T. Winters, T. Manshreck, H. Wright. — O’Reilly Media, 2020. — ISBN 978-1-492-08279-8. — URL: https://github.com/abseil/abseil.github.io/blob/master/resources/swe-book/html/ch11.html (дата обращения: 21.10.2026). — Текст : электронный.

9. Zeller, A. The Debugging Book / A. Zeller. — 2021. — URL: https://github.com/uds-se/debuggingbook (дата обращения: 21.10.2026). — Текст : электронный.

10. The Fuzzing Book / A. Zeller, R. Gopinath, M. Böhme [et al.]. — 2021. — URL: https://github.com/uds-se/fuzzingbook (дата обращения: 21.10.2026). — Текст : электронный.

11. Python Software Foundation. Python 3.12 Documentation: sqlite3, decimal, unicodedata. — URL: https://github.com/python/cpython/tree/3.12/Doc/library (дата обращения: 21.10.2026). — Текст : электронный.

12. PostgreSQL Global Development Group. PostgreSQL 18 Documentation: Data Definition. — URL: https://github.com/postgres/postgres/blob/REL_18_STABLE/doc/src/sgml/ddl.sgml (дата обращения: 21.10.2026). — Текст : электронный.

13. FastAPI. Templates: официальная документация. — URL: https://github.com/fastapi/fastapi/blob/master/docs/en/docs/advanced/templates.md (дата обращения: 21.10.2026). — Текст : электронный.

14. SQLAlchemy. Session Basics: официальная документация. — URL: https://github.com/sqlalchemy/sqlalchemy/blob/main/doc/build/orm/session_basics.rst (дата обращения: 21.10.2026). — Текст : электронный.

15. Jinja. Template Designer Documentation. — URL: https://github.com/pallets/jinja/blob/main/docs/templates.rst (дата обращения: 21.10.2026). — Текст : электронный.

16. Pydantic. Strict Mode: официальная документация. — URL: https://github.com/pydantic/pydantic/blob/main/docs/concepts/strict_mode.md (дата обращения: 21.10.2026). — Текст : электронный.

17. pytest. How to Use Fixtures: официальная документация. — URL: https://github.com/pytest-dev/pytest/blob/main/doc/en/how-to/fixtures.rst (дата обращения: 21.10.2026). — Текст : электронный.

18. Fielding, R. HTTP Semantics / R. Fielding, M. Nottingham, J. Reschke. — 2022. — RFC 9110. — URL: https://github.com/httpwg/http-core/blob/main/rfc9110.xml (дата обращения: 21.10.2026). — Текст : электронный.

19. Microsoft. Playwright for Python: Getting Started — Library. — URL: https://github.com/microsoft/playwright/blob/main/docs/src/library-python.md (дата обращения: 21.10.2026). — Текст : электронный.

20. Uvicorn. Settings: официальная документация. — URL: https://github.com/encode/uvicorn/blob/main/docs/settings.md (дата обращения: 21.10.2026). — Текст : электронный.

ПРИЛОЖЕНИЕ А. ЭКРАНЫ И РЕЗУЛЬТАТЫ ДЕМОНСТРАЦИИ

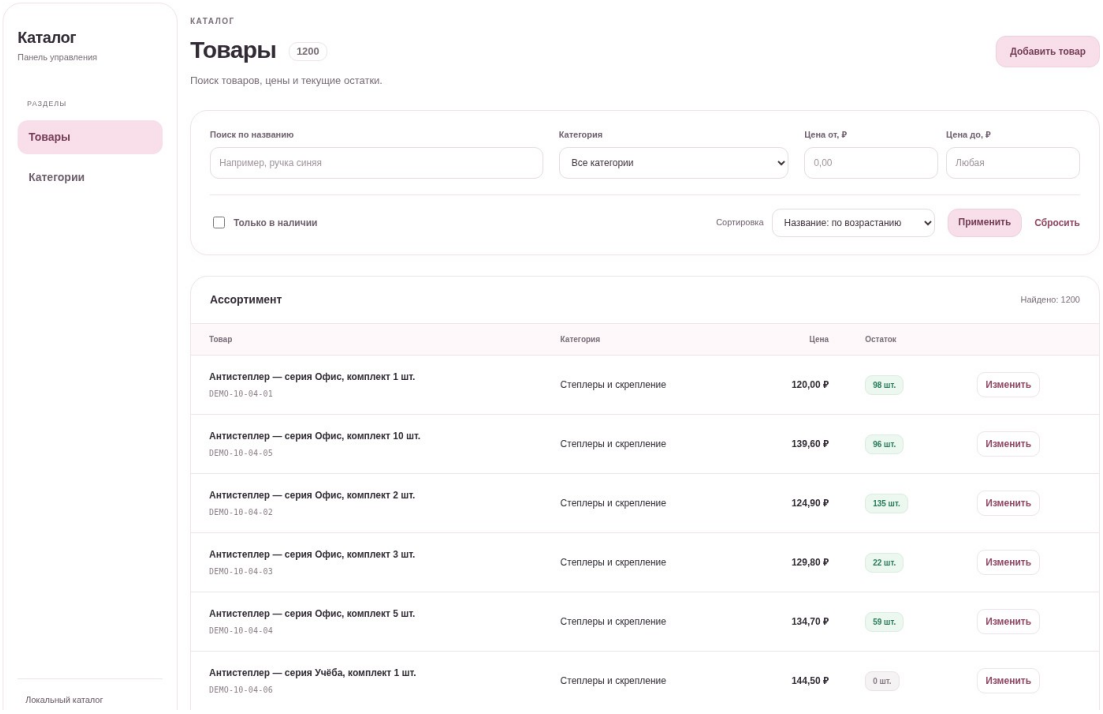


Рисунок А.1 — Каталог из 1200 демонстрационных товаров

Источник: снимок приложения при браузерной проверке 12.11.2026.

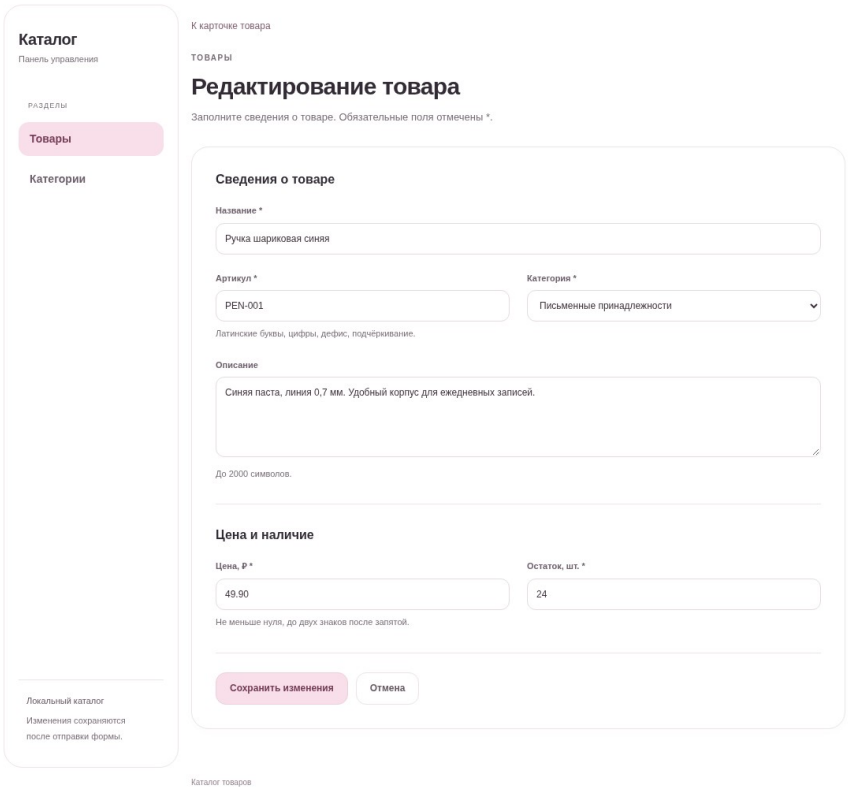


Рисунок А.2 — Форма редактирования товара

Источник: снимок приложения при браузерной проверке 12.11.2026.

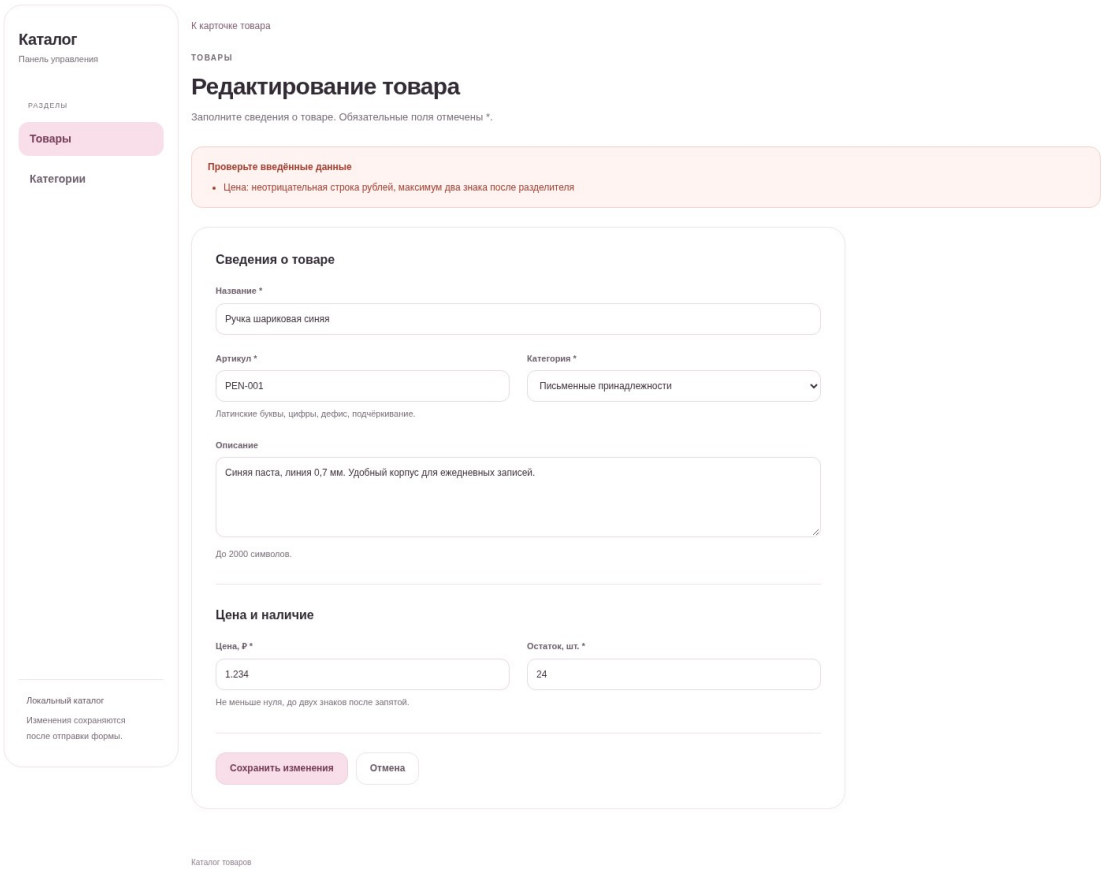


Рисунок А.3 — Сохранение ввода при неправильной цене

Источник: снимок приложения при браузерной проверке 12.11.2026.

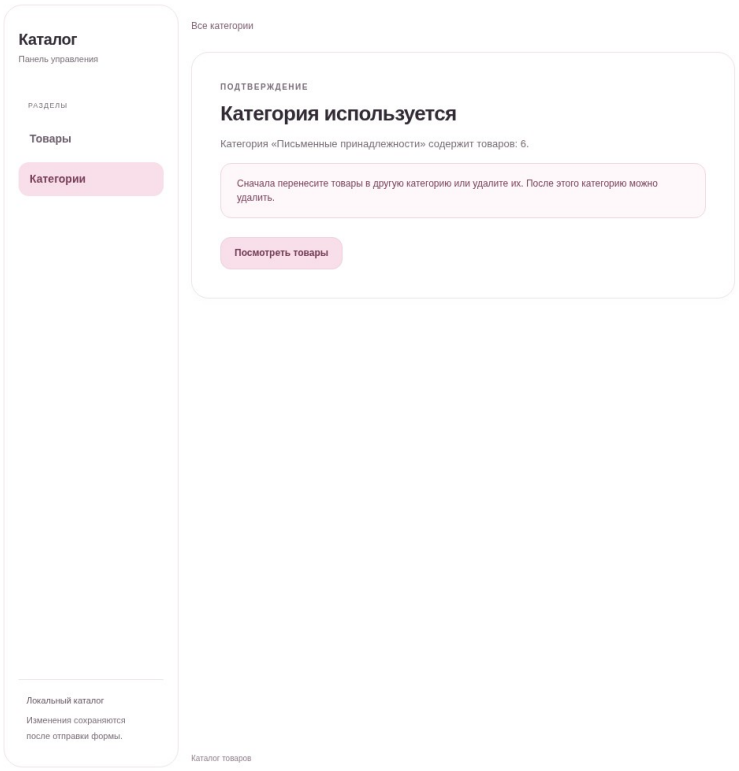


Рисунок А.4 — Запрет удаления категории с товарами

Источник: снимок приложения при браузерной проверке 12.11.2026.

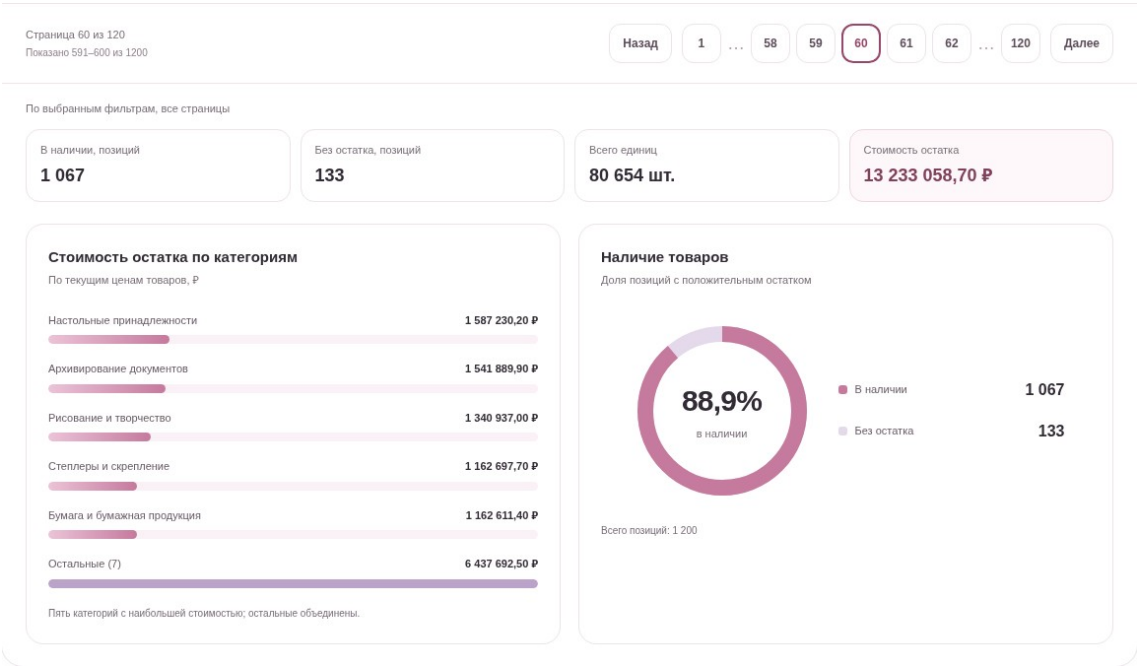


Рисунок А.5 — Сводные показатели и диаграммы каталога

Источник: снимок приложения при браузерной проверке 12.11.2026.